Lê Quốc Hưng N25DECFC081
F25CQCFO2-N

① P1: 2,1 | 2,1 P2: 2,5 P3: 2,5 P4: 2,5 9,6

Problem 1: Singly linked-list - In Place Reversal and Pointer tracing

Given a singly linked list: head → [10] → [25] → [40] → [15] → NULL.

- Task 1: Write a function/algorithm in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function.
- Task 2: Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed.

2,5 Problem 2 Circular linked-list - Insert at Head

Given a singly circular linked list: head → [10] → [20] → [30] → (points back to head)

- Task 1: Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at the



TTDOOK
Scanned with CamScanner

front of the circular linked list while maintaining its circular structure.

- Task 2: Trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into the given list.

2.5 Problem 3: Doubly Linked List - Insert before Target and Edge cases

Given a doubly linked list containing integers.

- Task 1: Write pseudocode for ~~InsertBefore~~ InsertBefore(head, target, newValue) to insert a new node containing newValue immediately ~~before~~ before the first occurence of target.
- Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links
- Task 3: Explicitly handle the following two edge cases in code/pseudocode
  1) Target is located at the head ~~of the node~~ (head)
  2) Target does not exist in the list.



TTBOOK

Scanned with CamScanner

2.5 Problem 4: Sorted Linked List - In order insertion

Given a singly linked list sorted in ascending order head → [2] → [5] → [8] → NULL.

- Task 1. Write the function SortedInsert (head, val) to insert val into the list such that the ascending order is preserved.
- Task 2. Construct a step-by-step trace table detailing point iterations and condition evaluations when inserting val = 6 into the list.

SOLUTION

2.1 Problem 1:

1.1 Task 4:

```
Reverse (head).
        prev = NULL
        cur = head
        next = NULL
while (cur != NULL)
        next = cur.next
        cur.next = prev
```



TTBOOK

Scanned with CamScanner

prev = curr

curr = next

head = prev

return head.

1.0 Task 2:

| Iteration | prev | cur | next | List direction |
|---|---|---|---|---|
| Initial | ~~NULL~~ | ~~10~~ | ~~next~~ | 10 → 25 → 40 → 15 → NULL |
| 1 | 10 | 25 | 25 | 10 → NULL ~~changed~~ changed ~~10 → 25 → 40 → 15 → NULL~~ |
| 2 | 25 | 40 | 40 | 25 → 10 → NULL |
| 3 | 40 | 15 | 15 | 40 → 25 → 10 → NULL |
| 4 | 15 | NULL | NULL | 15 → 40 → 25 → 10 → NULL |

Final result: ?

Problem 2:

Task 1.



TTBOOK

Scanned with CamScanner

(2)

Họ và tên: Phương năm sinh: 2005 DE25CQCE02-N

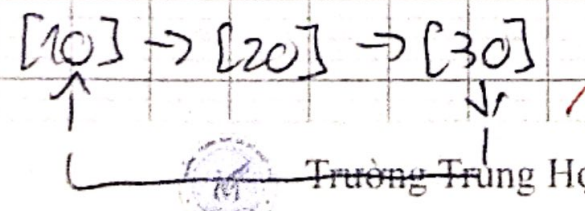
(2) Task 1: 1,25

```
Insert Head (Head, newValue):
    newNode ← new Node (newValue)
    if head = NULL:
        newNode.next ← newNode
        head ← newNode
        return head
    tail ← head
    while (tail.next ≠ head):
        tail ← tail.next
    newNode.next ← head
    tail.next ← newNode
    head ← newNode

    return head
```

1,25 Task 2:

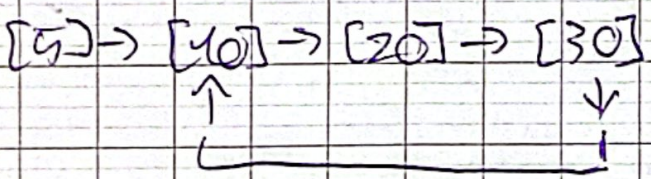
Initial:




Trường Trung Học Cơ Sở Nhựt Tảo

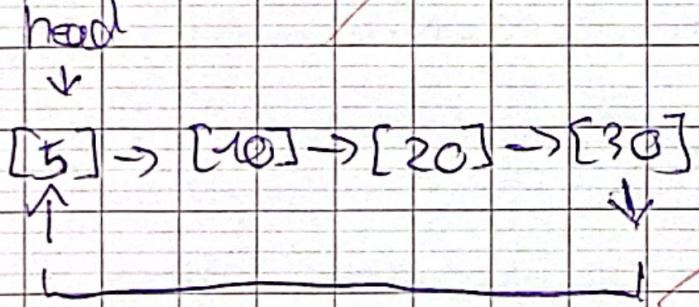
Scanned with CamScanner

- Find the tail:
  tail = [30]
- Create new node.
  [5]
- Set:
  newNode.next = head.
- So:




- Then:
  tail.next = newNode
- Finally
  head = newNode

→




Problem ?:

Q: Tasby.



Scanned with CamScanner

```
Insert Before (head, target, newValue):
    cur ← head
while cur ≠ NULL AND cur.value ≠ target:
    cur ← cur.next
if cur = NULL:
    return head  // target does not exist.
newNode ← new Node (newValue).
if (cur = head):
    newNode.prev ← NULL
    newNode.next ← cur
    cur.prev ← newNode
    head ← newNode
    return head.
prevNode ← cur.prev

newNode.prev ← preNode
newNode.next ← cur.
prevNode.next ← cur.
cur.prev ← newNode

return head.
```



Scanned with CamScanner

Thứ ngày tháng năm

0,75 Task 2:

prevNode ← cur.prev

newNode.prev ← prevNode
newNode.next ← cur
prevNode.next ← newNode.
cur.prev ← newNode.

Before:
prevNode <-> cur

After:
prevNode <-> newNode <-> cur.

Task 3: Works because newNode is connected to both neighboring nodes before the old connection is fully replaced.

1,0 Task 3: Edge cases



Scanned with CamScanner

Lê Quốc Thắng N25DCEO84
E25CQCE02-N.

(3)

Case 1:

head -> [10] <-> [20] <-> [30]

newNode.prev ← NULL.

newNode.next ← head

head.prev ← newNode

head ← newNode



head

↓

[5] <-> [10] <-> [20] <-> [30]

Case 2:

- If

cur = NULL.

- Then:

return head



Problem 4:

1,25 Task 4:



TTBOOK

Scanned with CamScanner

```
// Sorted Insert (head, val):
newNode ← new Node (val)

if head = NULL OR val < head.value:
    newNode.next ← head
    head ← newNode
    return head
cur ← head
while (cur.next ≠ NULL AND cur.next.value
< val): cur ← cur.next

newNode.next ← cur.next
cur.next ← newNode
return head
```

1,25 Task 2

| Step | cur | Condition | Action |
|---|---|---|---|
| Initial | 2 | 5 < 6 → TRUE | Move cur |
| 1 | 5 | 8 < 6 → FALSE | Stop |
| Insert | 5 | — | Insert 6 after 5 |

Một trang vở một tương lai
ITBOOK
Scanned with CamScanner

- Start: cur = [2]
- Check: cur.next.value < 6
  5 < 6 → TRUE
- Move. cur = [5]
- Check. cur.next.value < 6
  8 < 6 → FALSE
- Stop
- Insert 6 between 5 and 8.

- Pointer assignments: newNode.next = cur.next
- So: [6] → [8]
- Then cur.next = newNode

Result:

head → [2] → [5] → [6] → [8] → NULL



ITBOOK

Scanned with CamScanner